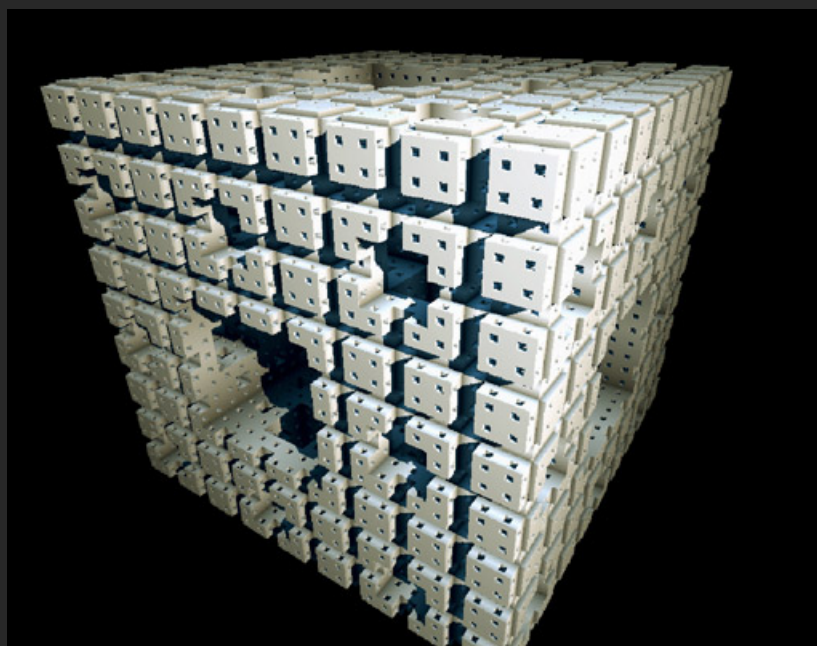


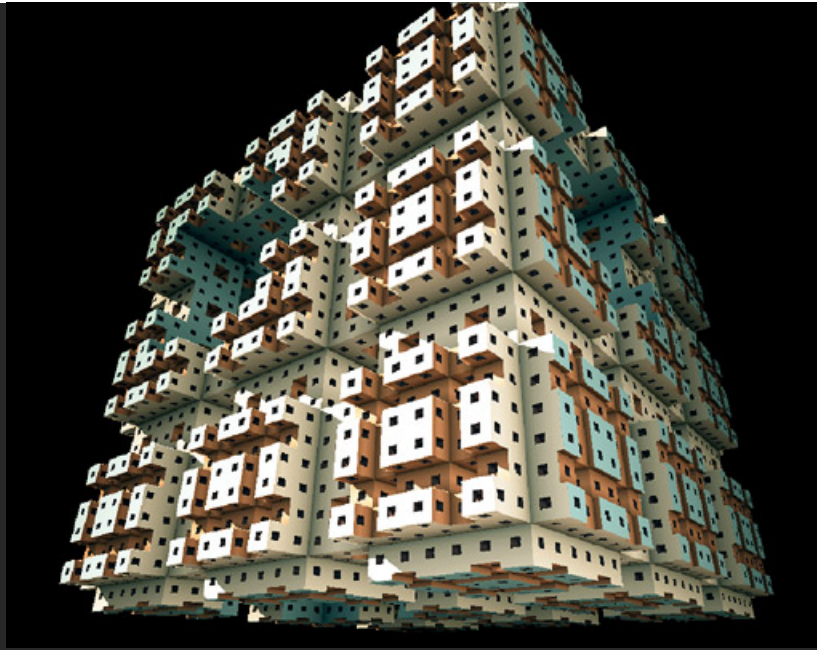
Inigo Quilez :: articles :: menger fractal - 2011



Intro

Producing simple analytic shapes with for a raymarching engine is very simple, as seen in the in this [reference article](#) in this very website. Besides simple combination of shapes to construct more complex compound shapes, there is also the possibility to do this composition algorithmically. Probably the most basic way of algorithmic composition, is the recursive introduction of regular smaller details. This naturally produces classic Cantor fractals. A good example of that is the "untraceable" 1 kilobyte demo by TBC.



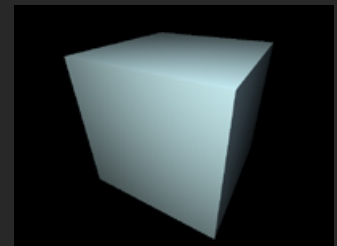


Probably the most trivial example is that of the **Menger Spone**. Let's take this as an example and do some raymarching experiment. You can see the final result at the end of this page. I assume you already have some code working, and that you must have your basic raymarching intersection routine look something like this:

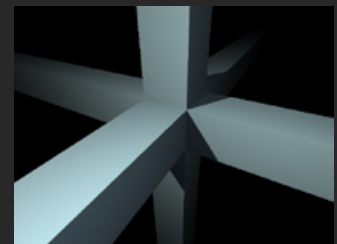
```
vec3 intersect( in vec3 ro, in vec3 rd )
{
    for(float t=0.0; t<10.0; )
    {
        vec3 h = map(ro + rd*t);
        if( h.x<0.001 )
            return vec3(t,h.yz);
        t += h;
    }
    return vec3(-1.0);
}
```

We will now define the map() function. We start by computing the distance to the unit cube:

```
vec3 map( in vec3 p )
{
    float d = sdBox(p,vec3(1.0));
    return vec3(d,0.0,0.0);
}
```



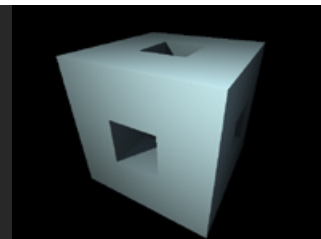
sdBox()



sdCross()

We now construct a cross made of 3 infinite boxes, by doing their union:

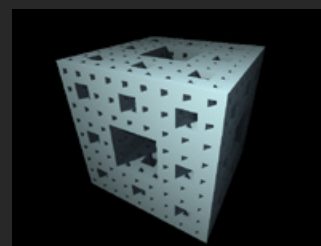
```
float sdCross( in vec3 p )
{
    float da = sdBox(p.xyz,vec3(1.0,1.0,1.0));
    float db = sdBox(p.yzx,vec3(1.0,1.0,1.0));
    float dc = sdBox(p.zxy,vec3(1.0,1.0,1.0));
    return min(da,min(db,dc));
}
```



sdBox() - sdCross()

This can be optimized by using 2D boxes instead:

```
float sdCross( in vec3 p )
{
    float da = sdBox(p.xy,vec2(1.0));
    float db = sdBox(p.yz,vec2(1.0));
    float dc = sdBox(p.zx,vec2(1.0));
    return min(da,min(db,dc));
}
```



sdBox() - iterate(sdCross())

Now we scale the cross by one third, and perform the subtraction of the cross to the box

```
vec4 map( in vec3 p )
{
    float d = sdBox(p,vec3(1.0));
    float c = sdCross(p*3.0)/3.0;
    d = max( d, -c );
    return vec4(d,1.0,1.0,1.0);
}
```

Finally, we can do this subtraction many times, iteratively:

```
vec3 map( in vec3 p )
{
    float d = sdBox(p,vec3(1.0));

    float s = 1.0;
    for( int m=0; m<3; m++ )
    {
        vec3 a = mod( p*s, 2.0 )-1.0;
        s *= 3.0;
        vec3 r = 1.0 - 3.0*abs(a);

        float c = sdCross(r)/s;
    }
}
```

```

    d = max(d,c);
}

return vec3(d,0.0,0.0);
}

```

For the sdCross() function, we can further optimize by noticing that we only care about the negative values of the 2d box functions. Therefore,

```

float sdCross( in vec3 p )
{
    float da = maxcomp(abs(p.xy));
    float db = maxcomp(abs(p.yz));
    float dc = maxcomp(abs(p.zx));
    return min(da,min(db,dc))-1.0;
}

```

or even better,

```

vec3 map( in vec3 p )
{
    float d = sdBox(p,vec3(1.0));

    float s = 1.0;
    for( int m=0; m<3; m++ )
    {
        vec3 a = mod( p*s, 2.0 )-1.0;
        s *= 3.0;
        vec3 r = abs(1.0 - 3.0*abs(a));

        float da = max(r.x,r.y);
        float db = max(r.y,r.z);
        float dc = max(r.z,r.x);
        float c = (min(da,min(db,dc))-1.0)/s;

        d = max(d,c);
    }

    return vec3(d,1.0,1.0);
}

```

Last step is, to compute the material id based on the iteration count, and some fake occlusion too (see images on the right of the article to see how it looks like when all these elements are put together):

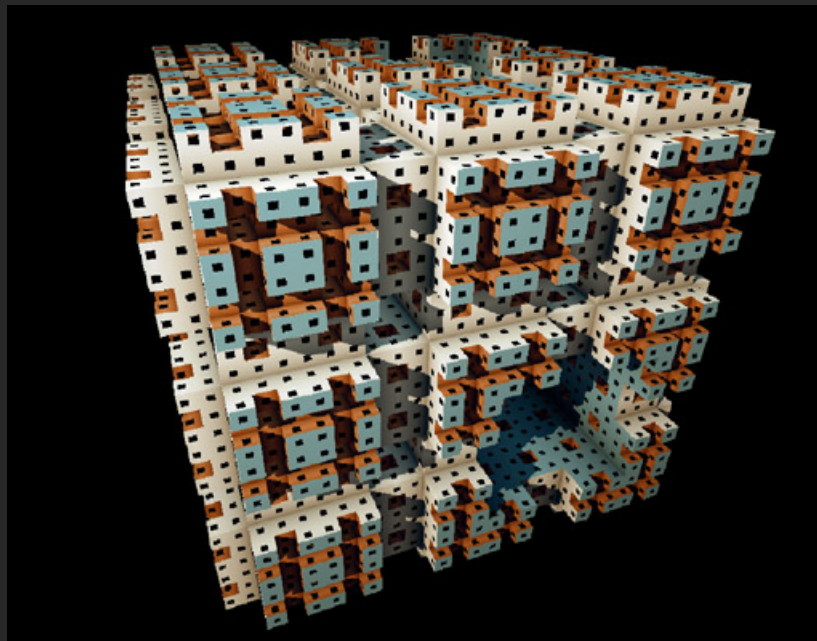
```
vec3 map( in vec3 p )
{
    float d = sdBox(p,vec3(1.0));
    vec3 res = vec3( d, 1.0, 0.0, 0.0 );

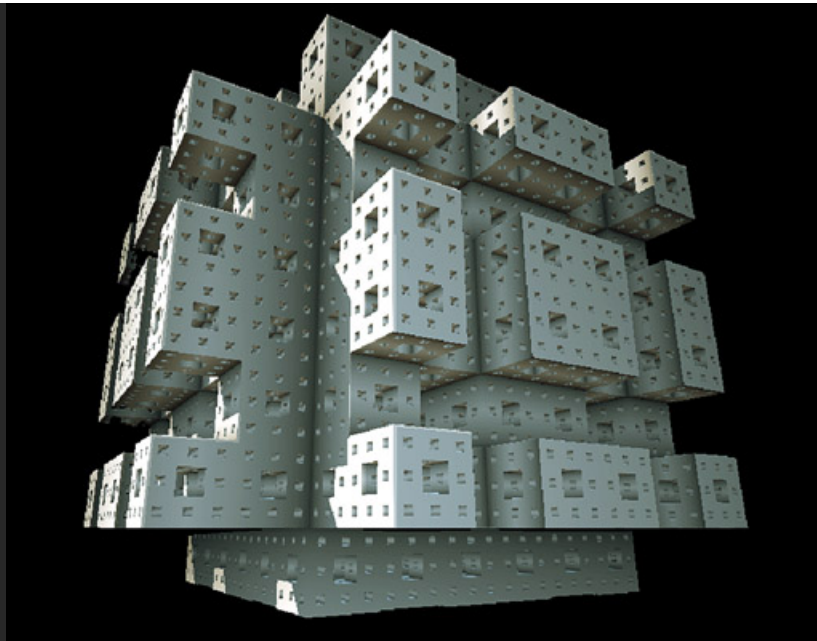
    float s = 1.0;
    for( int m=0; m<3; m++ )
    {
        vec3 a = mod( p*s, 2.0 )-1.0;
        s *= 3.0;
        vec3 r = abs(1.0 - 3.0*abs(a));

        float da = max(r.x,r.y);
        float db = max(r.y,r.z);
        float dc = max(r.z,r.x);
        float c = (min(da,min(db,dc))-1.0)/s;

        if( c>d )
        {
            d = c;
            res = vec3( d, 0.2*da*db*dc, (1.0+float(m))/4.0, 0.0 );
        }
    }

    return res;
}
```





Some neat tricks can be applied during the iterative subtraction of cubes, such as translating and rotating the point p a little bit in each iteration. That produces less symmetrical patterns, as can be seen in the images to the right of this article or here below (which was rendered with a simple [pathtracer](#)):



And finally, this is the source code for a reference implementation, rendered in realtime (click the play button

to see it move, and the title in the image to jump to the source code):



[inigo quilez](#) - learning computer graphics since 1994